

RSA (Rivest-Shamir-Adleman)

Aim

To implement RSA, a public-key cipher whose security rests on the difficulty of factoring the product of two large primes. A keypair is derived from two primes p and q : the public key (e, n) is shared, the private key (d, n) is kept secret. A message is encrypted with the public key as $c = m^e \bmod n$ and only the matching private key recovers it as $m = c^d \bmod n$. This demonstrator works on real integer arithmetic (BigInt) and encrypts one character code point per block.

Algorithm

Key generation is a one-time setup that produces the keypair used by encrypt and decrypt.

```
1 Function KeyGen( $p, q$ ):
2    $n \leftarrow p * q$ 
3    $\phi \leftarrow (p - 1) * (q - 1)$                                 // Euler's totient of  $n$ 
4   choose  $e$  with  $1 < e < \phi$  and  $\gcd(e, \phi) = 1$                 // public exponent
5    $d \leftarrow$  modular inverse of  $e \bmod \phi$                       // via extended Euclid
6   return public  $(e, n)$  and private  $(d, n)$ 
```

```
1 Function Encrypt( $plain\_text, e, n$ ):
2   cipher  $\leftarrow$  empty list
3   foreach character  $ch$  in  $plain\_text$  do
4      $m \leftarrow$  code point of  $ch$ 
5     if  $m \geq n$  then
6       error                                                    // block too large; use bigger primes
7     append  $(m$  raised to  $e) \bmod n$  to cipher                    // fast modular exponentiation
8   return cipher
```

```
1 Function Decrypt( $cipher, d, n$ ):
2   plain  $\leftarrow$  ""
3   foreach value  $c$  in  $cipher$  do
4      $m \leftarrow (c$  raised to  $d) \bmod n$ 
5     append character with code point  $m$  to plain
6   return plain
```

Output

Key generation — primes $p = 17, q = 11$

Quantity	Computation	Value
n	$p * q = 17 * 11$	187
ϕ	$(p - 1)(q - 1) = 16 * 10$	160
e	smallest coprime with ϕ (> 1)	17
d	inverse of $e \bmod \phi$	113

Check: $e * d \bmod \phi = 17 * 113 \bmod 160 = 1$, so encryption and decryption are inverses. Public key = (17, 187), private key = (113, 187).

PUBLIC-KEY CRYPTOGRAPHY · WORKED BY HAND

Two primes go in. A lock and its only key come out.

RSA's whole secret is a one-way street: multiplying $p \cdot q$ is easy, factoring the result back apart is not. Pick two primes below and watch the keypair get *derived*, digit by digit — then send a message down the public lane and read it back with the private one.

A THE FORGE

DERIVE KEYS FROM PRIMES

B THE CHANNEL

ENCRYPT, THEN RECOVER

prime p

61

prime q

53

public exponent e (optional — auto-chosen if blank)

17

Forge keypair

Roll fresh primes

$p \cdot q$ modulus n

$61 \cdot 53 = 3233$

$\phi(n)$ $(p-1)(q-1)$

$(60)(52) = 3120$

$e \text{ gcd}(e, \phi) = 1$

17

$d \text{ } e \cdot d \equiv 1 \pmod{\phi}$

2753

public key share freely

$(e = 17, n = 3233)$

private key keep secret

$(d = 2753, n = 3233)$

Seal a message

PUBLIC LANE · ANYONE CAN ENCRYPT WITH (E, N)

plaintext

meet at dawn

Encrypt →

2271 1313 1313 884 1992 1632 884 1992 1773 1632 1107 2235

Open the message

PRIVATE LANE · ONLY (D, N) CAN DECRYPT

← Decrypt with private key

meet at dawn

Figure 1: RSA demonstrator

Encrypt — message **Hi** with public key (**e** = 17, **n** = 187)

Each character's code point is raised to **e** mod **n**:

Char	Code (m)	$c = (m \text{ raised to } e) \bmod n$	Cipher
H	72	$(72 \text{ raised to } 17) \bmod 187$	140
i	105	$(105 \text{ raised to } 17) \bmod 187$	173

Result: **Hi** -> [140, 173]

Decrypt — cipher [140, 173] with private key (**d** = 113, **n** = 187)

Each cipher value is raised to **d** mod **n** to recover the code point:

Cipher (c)	$m = (c \text{ raised to } d) \bmod n$	Code	Char
140	$(140 \text{ raised to } 113) \bmod 187$	72	H
173	$(173 \text{ raised to } 113) \bmod 187$	105	i

Result: [140, 173] -> **Hi**

Every character code must be smaller than **n**; the primes here (**n** = 187) only fit code points below 187, so real use needs far larger primes. An eavesdropper who sees (**e**, **n**) and the cipher would have to factor **n** back into **p** and **q** to find **d**, which is infeasible for large primes.